
Back to Basics: Let Denoising Generative Models Denoise

CVPR 2026
citations 238

Department of Computer Engineering and Artificial Intelligence
MAI Lab.

YS Park

Back to Basics: Let Denoising Generative Models Denoise

Tianhong Li, Kaiming He; Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2026, pp. 36115-36125

Abstract

Today's denoising diffusion models do not "denoise" in the classical sense, i.e., they do not directly predict clean images. Rather, the neural networks predict noise or a noised quantity. In this paper, we suggest that predicting clean data and predicting noised quantities are fundamentally different. According to the manifold assumption, natural data should lie on a low-dimensional manifold, whereas noised quantities do not. With this assumption, we advocate for models that directly predict clean data, which allows apparently under-capacity networks to operate effectively in very high-dimensional spaces. We show that simple, large-patch Transformers on pixels can be strong generative models: using no tokenizer, no pre-training, and no extra loss. Our approach is conceptually nothing more than "Just image Transformers", or JiT, as we call it. We report competitive results using JiT with large patch sizes of 16 and 32 on ImageNet at resolutions of 256 and 512, where predicting high-dimensional noised quantities can fail catastrophically. With our networks mapping back to the basics of the manifold, our research goes back to basics and pursues a self-contained paradigm for Transformer-based diffusion on raw natural data.

- History of Generative AI, Then and now ...
- Introduction
- On Prediction Outputs of Diffusion Models
- “Just Image Transformers” for Diffusion
- Comparisons
- Discussion and conclusion

생성 모델이란 목표 도메인의 데이터를 생성하는 모델을 말한다. 그리고 몇몇 생성 모델은 주어진 데이터

x 의 우도 $p(x)$ 를 평가할 수도 있다. 데이터가 어떻게 생성되는지 이해하는 것은 **그 데이터를 이해하는** 효과적인 방법의 하나이며, 또한 데이터를 자유롭게 생성할 수 있다면 많은 애플리케이션에 도움이 된다.

그래서 오래전부터 생성 모델에 관해 많은 연구가 이루어져 왔다.

- 확산 모델의 수학, 요카노하라 다이스케 中...

History of Generative AI, Then and now ...

2013

Auto-Encoding Variational Bayes

Diederik P. Kingma
 Machine Learning Group
 Universiteit van Amsterdam
 dpkingma@gmail.com

Max Welling
 Machine Learning Group
 Universiteit van Amsterdam
 welling.max@gmail.com

Abstract

How can we perform efficient inference and learning in directed probabilistic models, in the presence of continuous latent variables with intractable posterior distributions, and large datasets? We introduce a stochastic variational inference and learning algorithm that scales to large datasets and, under some mild differentiability conditions, even works in the intractable case. Our contributions are two-fold. First, we show that a reparameterization of the variational lower bound yields a lower bound estimator that can be straightforwardly optimized using standard stochastic gradient methods. Second, we show that for i.i.d. datasets with continuous latent variables per datapoint, posterior inference can be made especially efficient by fitting an approximate inference model (also called a recognition model) to the intractable posterior using the proposed lower bound estimator. Theoretical advantages are reflected in experimental results.

2014

Generative Adversarial Nets

Ian J. Goodfellow,¹ Jean Pouget-Abadie,² Mehdi Mirza, Bing Xu, David Warde-Farley,
 Sherjil Ozair,¹ Aaron Courville, Yoshua Bengio¹
 Département d'informatique et de recherche opérationnelle
 Université de Montréal
 Montréal, QC H3C 3J7

Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D , a unique solution exists, with G recovering the training data distribution and D equal to $\frac{1}{2}$ everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.

2020

Denoising Diffusion Probabilistic Models

Jonathan Ho
 UC Berkeley
 jonathanho@berkeley.edu

Ajay Jain
 UC Berkeley
 ajayj@berkeley.edu

Pieter Abbeel
 UC Berkeley
 pabbeel@cs.berkeley.edu

Abstract

We present high quality image synthesis results using diffusion probabilistic models, a class of latent variable models inspired by considerations from nonequilibrium thermodynamics. Our best results are obtained by training on a weighted variational bound designed according to a novel connection between diffusion probabilistic models and denoising score matching with Langevin dynamics, and our models naturally admit a progressive lossy decomposition scheme that can be interpreted as a generalization of autoregressive decoding. On the unconditional CIFAR10 dataset, we obtain an Inception score of 9.46 and a state-of-the-art FID score of 3.17. On 256x256 LSUN, we obtain sample quality similar to ProgressiveGAN. Our implementation is available at <https://github.com/hojonathanho/diffusion>.

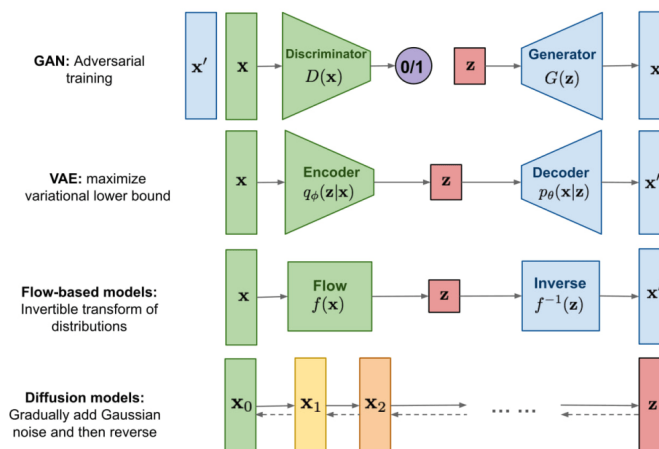
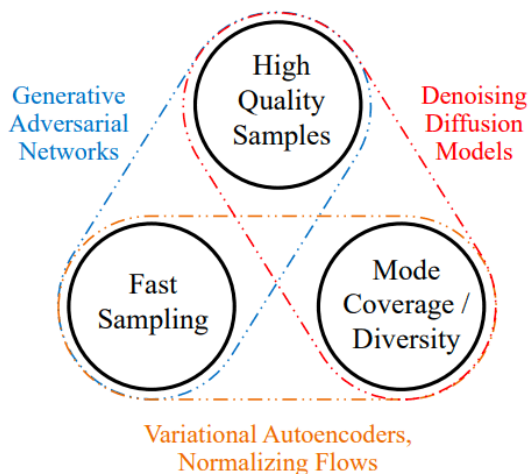
2022

FLOW MATCHING FOR GENERATIVE MODELING

Yaron Lipman^{1,2} Ricky T. Q. Chen¹ Heli Ben-Hamu² Maximilian Nickel¹ Matt Le¹
¹Meta AI (FAIR) ²Weizmann Institute of Science

ABSTRACT

We introduce a new paradigm for generative modeling built on Continuous Normalizing Flows (CNFs), allowing us to train CNFs at unprecedented scale. Specifically, we present the notion of Flow Matching (FM), a simulation-free approach for training CNFs based on regressing vector fields of fixed conditional probability paths for transforming between noise and data samples—which subsumes existing diffusion paths as specific instances. Interestingly, we find that employing FM with diffusion paths results in a more robust and stable alternative for training diffusion models. Furthermore, Flow Matching opens the door to training CNFs with other, non-diffusion probability paths. An instance of particular interest is using Optimal Transport (OT) displacement interpolation to define the conditional probability paths. These paths are more efficient than diffusion paths, provide faster training and sampling, and result in better generalization. Training CNFs using Flow Matching on ImageNet leads to consistently better performance than alternative diffusion-based methods in terms of both likelihood and sample quality, and allows fast and reliable sample generation using off-the-shelf numerical ODE solvers.



Paradigm	Quality	Diversity	Speed
VAE	✗	✓	✓
GAN	✓	✗	✓
Diffusion	✓	✓	✗

- Diffusion generative model이 처음 개발 됐을 때는 **denoising**하도록 설계 되었음.
 - Corrupted image에서 clean image 예측(x -prediction) 함.
- 그러나 DDPM에서는 noise를 예측(ϵ -prediction)하거나 flow-matching과 결합하여 flow velocity를 예측 (v -prediction, a quantity that combines clean data and noise) 하는 식으로 발전하고 있음.
 - 즉, 최근 diffusion model은 noise나 noise의 양을 예측하는 모델임.
- x -prediction, ϵ -prediction, v -prediction은 밀접한 관계가 있기에 지금까지는 **network가 정확히 무엇을 직접 예측해야 하는지**에 대해서는 상대적으로 많은 관심이 주어지지 않았다.
 - 그러나, x -prediction과 ϵ -, v -prediction은 데이터가 존재하는 공간의 관점에서 **본질적으로 다른 문제임**.

$$224 \times 224 \times 3 = 150,528$$

$$32 \times 32 \times 4 = 4,096 \text{ (자급자족X)}$$

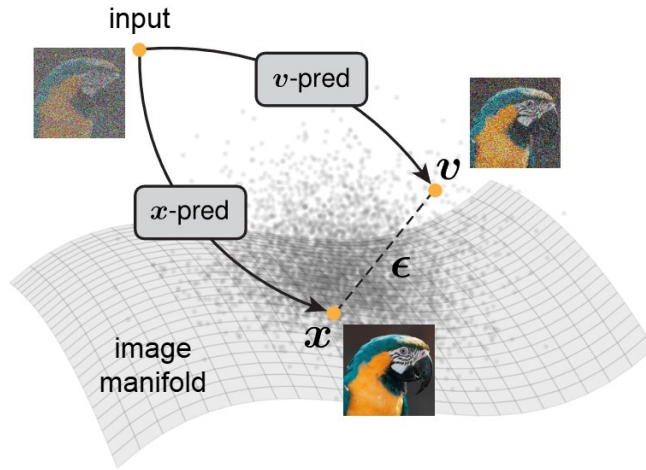
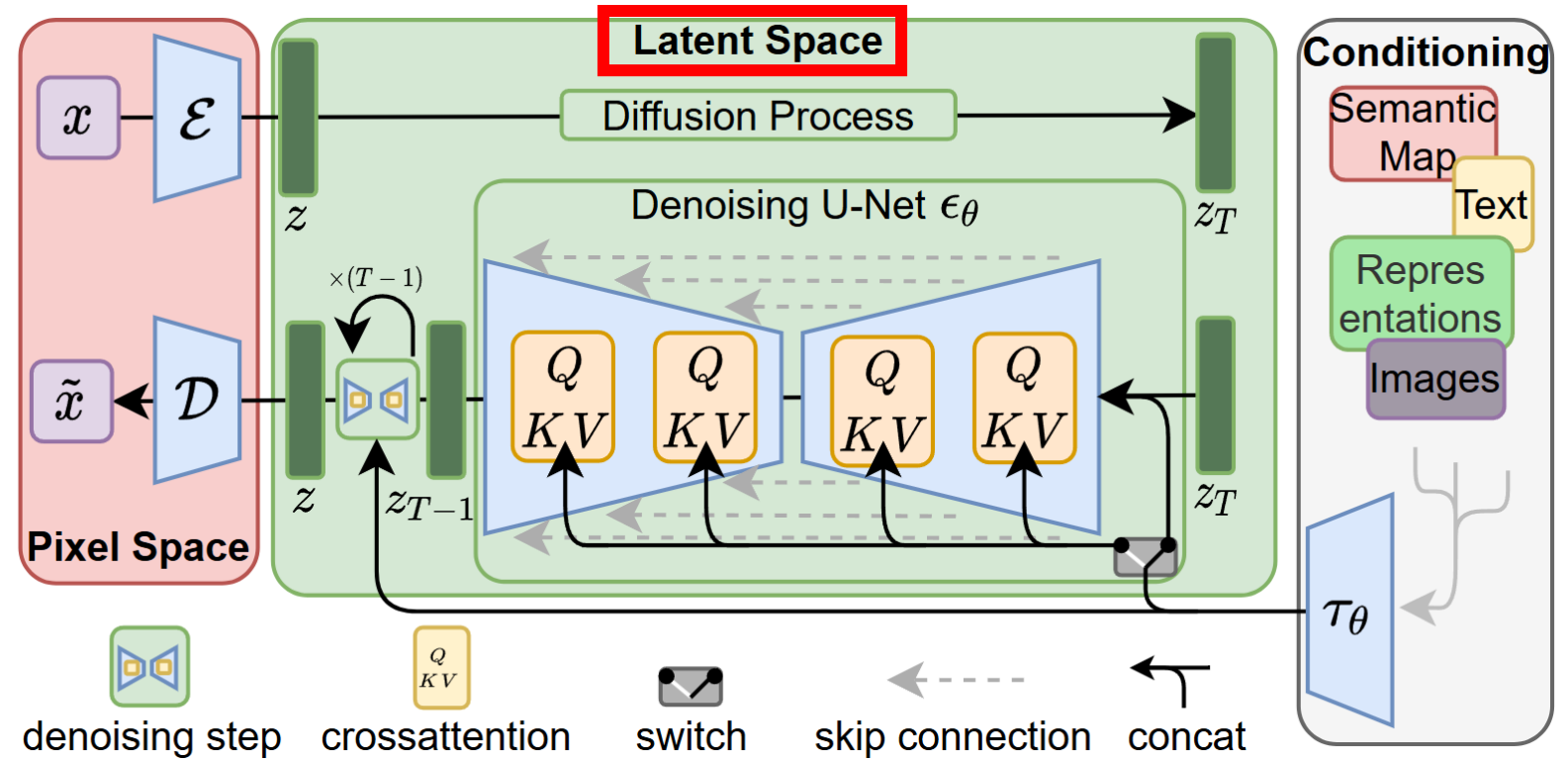


Figure 1. **The Manifold Assumption** [4] hypothesizes that natural images lie on a low-dimensional manifold within the high-dimensional pixel space. While a clean image x can be modeled as on-manifold, the noise ϵ or flow velocity v (e.g., $v = x - \epsilon$) is inherently off-manifold. Training a neural network to predict a clean image (i.e., x -prediction) is *fundamentally different* from training it to predict noise or a noised quantity (i.e., ϵ/v -prediction).



- 자급자족을 위해 pixel-level도 많은 발전을 했으나, dense convolutions, smaller patches, increasing channels, long skip connection 등의 methods를 활용하여 information bottleneck을 우회함.
- 본 논문에서는 x -prediction을 이용하여 **raw pixels로 구성된 큰 이미지 패치**를 입력으로 사용하는 단순한 **Vision Transformer(ViT)**만으로도 diffusion modeling을 효과적으로 수행할 수 있음을 보임.
 - NO Latent tokenizer
 - NO Adversarial loss
 - NO Perceptual loss
 - NO pre-trained classifier
 - NO Representation alignment
 - NO self-supervised pre-training
 - Nothing more than **Just image Transformer (JiT)**

- Diffusion model은 x, ϵ, v 공간에서 상미분방정식(ODE)로 공식화할 수 있음. 그러나 loss space와 output space가 같을 필요는 없음!

Data distribution, $x \sim p_{data}(x)$ noise distribution $\epsilon \sim p_{noise}(\epsilon)$ e. g. $\epsilon \sim \mathcal{N}(0, I)$

$$z_t = a_t x + b_t \epsilon = tx + (1 - t)\epsilon$$

$$v_t = z'_t = a'_t x + b'_t \epsilon$$

$$v = x - \epsilon$$

$$L = \mathbb{E}_{t,x,\epsilon} \| v_\theta(z_t, t) - v \|^2$$

On Prediction Outputs of Diffusion Models

Prediction Space and Loss Space

- Diffusion model은 x, ϵ, v 공간에서 상미분방정식(ODE)로 공식화할 수 있음. 그러나 loss space와 output space가 같을 필요는 없음!

$$\begin{cases} x_\theta = \text{net}_\theta(z_t, t) \\ z_t = tx_\theta + (1-t)\epsilon_\theta \\ v_\theta = x_\theta - \epsilon_\theta \end{cases}$$

	(a) x -pred $x_\theta := \text{net}_\theta(z_t, t)$	(b) ϵ -pred $\epsilon_\theta := \text{net}_\theta(z_t, t)$	(c) v -pred $v_\theta := \text{net}_\theta(z_t, t)$
(1) x -loss: $\mathbb{E}\ x_\theta - x\ ^2$	x_θ	$x_\theta = (z_t - (1-t)\epsilon_\theta)/t$	$x_\theta = (1-t)v_\theta + z_t$
(2) ϵ -loss: $\mathbb{E}\ \epsilon_\theta - \epsilon\ ^2$	$\epsilon_\theta = (z_t - tx_\theta)/(1-t)$	ϵ_θ	$\epsilon_\theta = z_t - tv_\theta$
(3) v -loss: $\mathbb{E}\ v_\theta - v\ ^2$	$v_\theta = (x_\theta - z_t)/(1-t)$	$v_\theta = (z_t - \epsilon_\theta)/t$	v_θ

On Prediction Outputs of Diffusion Models

Prediction Space and Loss Space

$$\begin{cases} x_\theta = \text{net}_\theta(z_t, t) \\ z_t = tx_\theta + (1-t)\epsilon_\theta \\ v_\theta = x_\theta - \epsilon_\theta \end{cases}$$

$$\epsilon_\theta = \frac{z_t - tx_\theta}{1-t}$$

$$\begin{aligned} v_\theta &= x_\theta - \frac{z_t - tx_\theta}{1-t} \\ &= \frac{(x_\theta - tx_\theta) - (z_t - tx_\theta)}{1-t} \\ &= \frac{x_\theta - z_t}{1-t} \end{aligned}$$

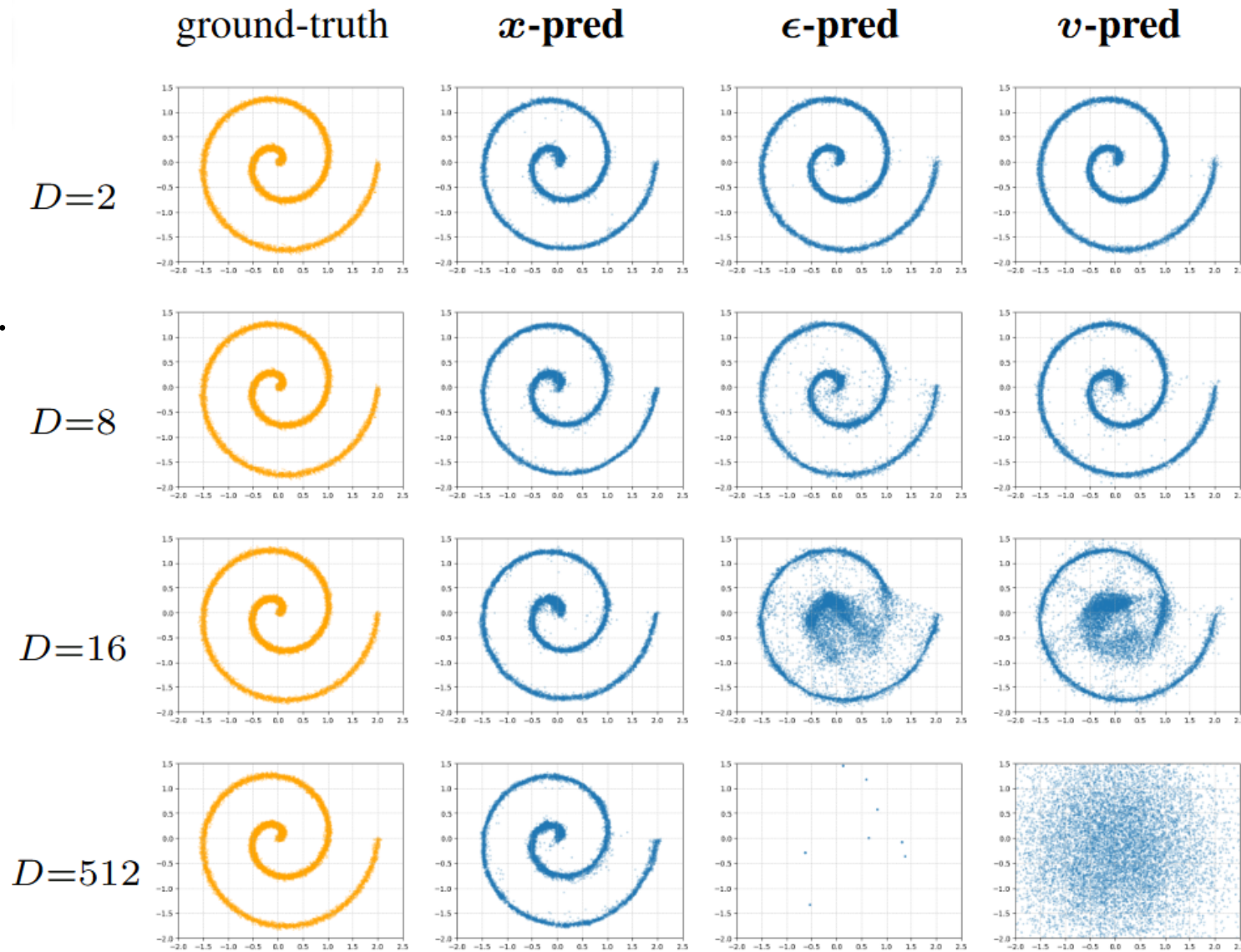
$$\begin{aligned} L &= \mathbb{E} \|v_\theta(z_t, t) - v\|^2 \\ &= \mathbb{E} \left\| \left(\frac{x_\theta - z_t}{1-t} \right) - \left(\frac{x - z_t}{1-t} \right) \right\|^2 \\ &= \mathbb{E} \left\| \frac{1}{1-t} (x_\theta - x) \right\|^2 \\ &= \mathbb{E} \frac{1}{(1-t)^2} \|x_\theta - x\|^2 \\ &\neq \mathbb{E} \|x_\theta - x\|^2 \end{aligned}$$

따라서 같은 x -prediction이더라도, loss space에 따라서 weighting 효과를 볼 수도 있다.

On Prediction Outputs of Diffusion Models

Toy Experiment

- x : low-dim manifold
- ϵ, v : off-manifold
- 5-layer 256D + ReLU MLP
 - Idle output이 2D이기 때문에 x -prediction만 모든 D 에서 성공함.



- The core idea of ViT [13] is "Transformer on Patches(ToP)"
 - x, ϵ, v, z_t 는 전부 $H \times W \times C$ -dim image data
 - 각 patch는 $p \times p \times 3$ -dim vector이며 $H_p \times W_p$ sequence를 만듦.
 - Transformer의 Dim = 768 (base)
 - Patch = $16 \times 16 \times 3 = 768$ for 256×256 , $32 \times 32 \times 3 = 3072$ for 512×512

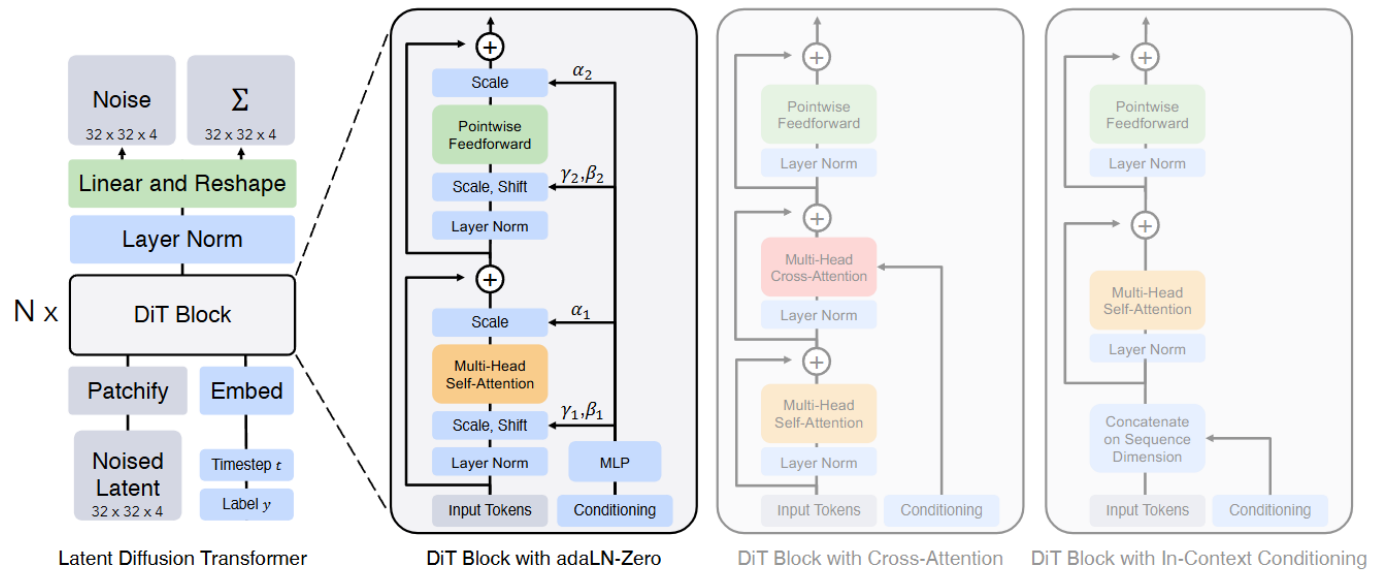
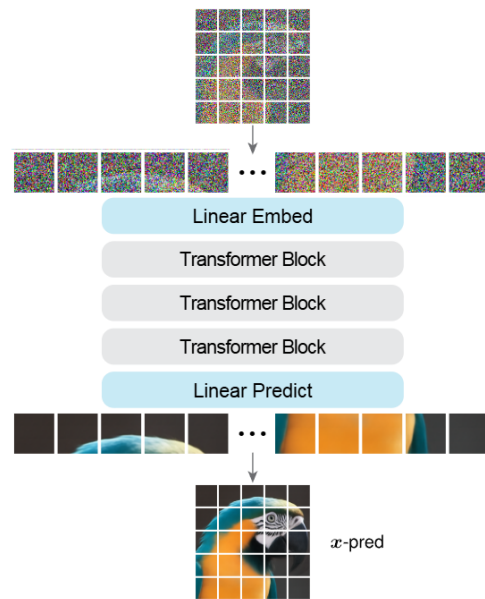


Figure 3. **The "Just image Transformer" (JiT) architecture:** simply a plain ViT [13] on patches of pixels for x -prediction.

Figure 3. **The Diffusion Transformer (DiT) architecture.** *Left:* We train conditional latent DiT models. The input latent is decomposed into patches and processed by several DiT blocks. *Right:* Details of our DiT blocks. We experiment with variants of standard transformer blocks that incorporate conditioning via adaptive layer norm, cross-attention and extra input tokens. Adaptive layer norm works best.

- **x -prediction is critical.**
 - 3가지 loss에서 전부 잘 작동함.
 - (a)에서 Patch와 model의 dim은 768로 같으나, position embedding을 처리하기 위한 추가적인 용량이 필요해보임.
 - (b) 처럼 patch dim이 48로 작은 경우엔 모두 성공적
- **Loss weighting is not sufficient.**
 - v -, ϵ -pred weighting과 무관하게 실패함.
 - x -pred, v -loss는 선호되는 것이지 필수는 아님.
 - v -loss: $\mathbb{E} \frac{1}{(1-t)^2} \|x_\theta - x\|^2$
 - ϵ -loss: $\mathbb{E} \frac{t^2}{(1-t)^2} \|x_\theta - x\|^2$

	x -pred	ϵ -pred	v -pred
x -loss	10.14	379.21	107.55
ϵ -loss	10.45	394.58	126.88
v -loss	8.62	372.38	96.53

(a) ImageNet 256×256, JiT-B/16

	x -pred	ϵ -pred	v -pred
x -loss	5.76	6.20	6.12
ϵ -loss	3.56	4.02	3.76
v -loss	3.55	3.63	3.46

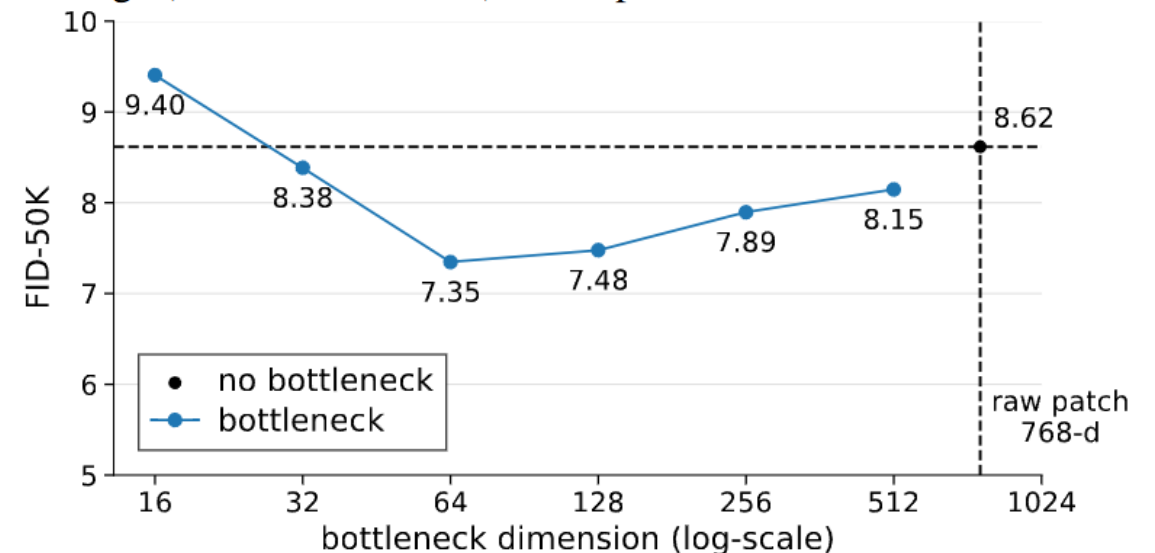
(b) ImageNet 64×64, JiT-B/4

Table 2. **Results of all combinations** of loss space and network space (see Tab. 1), evaluated by FID-50K on ImageNet: (a) JiT-B/16 at 256 resolution, 768-d per patch; (b) JiT-B/4 at 64 resolution, 48-d per patch. We annotate catastrophic failures in red and reasonable results by green. Settings: 200 epochs, with CFG [22].

- **Noise-level shift is not sufficient.**
 - noise를 조절해도 ϵ , v -pred의 실패는 해결되지 않음.
 - $\mu \downarrow \Rightarrow t \downarrow \Rightarrow z_t$ 가 더 noisy해짐.
- **Bottleneck can be beneficial.**
 - 오히려 bottleneck이 도움이 될 수 있음.

	t -shift (μ)	x -pred	ϵ -pred	v -pred
(lower noise)	0.0	14.44	464.25	120.03
	-0.4	9.79	372.91	109.93
	-0.8	8.62	372.36	96.53
(higher noise)	-1.2	8.99	355.25	106.85

Table 3. **Noise-level shift** (JiT-B/16, ImageNet 256×256 , FID-50K). We shift the noise level by adjusting μ in the logit-normal t -sampler [15]. An appropriate noise level is useful, but is not sufficient for addressing the catastrophic failure in ϵ -/ v -prediction. Settings (the same as Tab. 2): 200 epochs, with CFG.



- **Increasing hidden units is not necessary.**
 - patch 차원이 커져도 hidden dimension을 반드시 키울 필요는 없음.

$$L = \mathbb{E}_{t,x,\epsilon} \|v_\theta - v\|^2$$

$$v_\theta(z_t, t) = \frac{\text{net}_\theta(z_t, t) - z_t}{1 - t}$$

- **"Just Advanced" Transformers**
 - **JiT**
 - + SwiGLU
 - + RMSNorm
 - + RoPE
 - + qk-norm
 - + in-context class conditioning

	JiT-B/16	JiT-L/16
Baseline (SwiGLU, RMSNorm)	7.48 (6.32)	-
+ RoPE, qk-norm	6.69 (5.44)	-
+ in-context class tokens	5.49 (4.37)	3.39 (2.79)

Table 4. **"Just Advanced" Transformers** with *general-purpose* designs. All are **JiT/16** for ImageNet 256×256, with bottleneck patch embedding (128-d, Fig. 4), evaluated by FID-50K. Settings: 200 epochs, with CFG (and with CFG interval [33] in brackets).

resolution	model	len	patch dim	hidens	params	Gflops	FID
256×256	JiT-B/16	256	768	768	131	25	4.37
512×512	JiT-B/32	256	3072	768	133	26	4.64
1024×1024	JiT-B/64	256	12288	768	141	30	4.82

Table 5. **ImageNet 1024×1024 with JiT-B/64**. All entries have roughly the same number of parameters and compute. Settings: if not specified here, the same as Tab. 4 (all are with CFG interval).

256×256	200-ep	600-ep	512×512	200-ep	600-ep
JiT-B/16	4.37	3.66	JiT-B/32	4.64	4.02
JiT-L/16	2.79	2.36	JiT-L/32	3.06	2.53
JiT-H/16	2.29	1.86	JiT-H/32	2.51	1.94
JiT-G/16	2.15	1.82	JiT-G/32	2.11	1.78

Table 6. **Scalability on ImageNet 256×256 and 512×512**, evaluated by FID-50K. All models have the *same* sequence length of 16×16, and thus the models at 512 resolution have nearly the same compute as their 256 counterparts. Settings: the same as Tab. 5.

- High-resolution generation on pixels.

ImgNet 256×256	pre-training			params	Gflops	FID↓	IS↑
	token	perc.	self-sup.				
Latent-space Diffusion							
DiT-XL/2 [46]	SD-VAE	VGG	-	675+49M	119	2.27	278.2
SiT-XL/2 [40]	SD-VAE	VGG	-	675+49M	119	2.06	277.5
REPA [74], SiT-XL/2	SD-VAE	VGG	DINOv2	675+49M	119	1.42	305.7
LightningDiT-XL/2 [73]	VA-VAE	VGG	DINOv2	675+49M	119	1.35	295.3
DDT-XL/2 [71]	SD-VAE	VGG	DINOv2	675+49M	119	1.26	310.6
RAE [78], DiT ^{DH} -XL/2	RAE	VGG	DINOv2	839+415M	146	1.13	262.6
Pixel-space (non-diffusion)							
JetFormer [65]	-	-	-	2.8B	-	6.64	-
FractalMAR-H [36]	-	-	-	848M	-	6.15	348.9
Pixel-space Diffusion							
ADM-G [12]	-	-	-	554M	1120	4.59	186.7
RIN [28]	-	-	-	410M	334	3.42	182.0
SiD [25] , UViT/2	-	-	-	2B	555	2.44	256.3
VDM++, UViT/2	-	-	-	2B	555	2.12	267.7
SiD2 [26], UViT/2	-	-	-	N/A	137	1.73	-
SiD2 [26], UViT/1	-	-	-	N/A	653	1.38	-
PixelFlow [6], XL/4	-	-	-	677M	2909	1.98	282.1
PixNerd [70], XL/16	-	-	DINOv2	700M	134	2.15	297
JiT-B/16	-	-	-	131M	25	3.66	275.1
JiT-L/16	-	-	-	459M	88	2.36	298.5
JiT-H/16	-	-	-	953M	182	1.86	303.4
JiT-G/16	-	-	-	2B	383	1.82	292.6

ImgNet 512×512	pre-training			params	Gflops	FID↓	IS↑
	token	perc.	self-sup.				
<i>Latent-space Diffusion</i>							
DiT-XL/2 [46]	SD-VAE	VGG	-	675+49M	525	3.04	240.8
SiT-XL/2 [40]	SD-VAE	VGG	-	675+49M	525	2.62	252.2
REPA [74], SiT-XL/2	SD-VAE	VGG	DINOv2	675+49M	525	2.08	274.6
DDT-XL/2 [71]	SD-VAE	VGG	DINOv2	675+49M	525	1.28	305.1
RAE [78], DiT ^{DH} -XL/2	RAE	VGG	DINOv2	839+415M	642	1.13	259.6
<i>Pixel-space Diffusion</i>							
ADM-G [12]	-	-	-	559M	1983	7.72	172.7
RIN [28]	-	-	-	320M	415	3.95	216.0
SiD [25], UViT/4	-	-	-	2B	555	3.02	248.7
VDM++, UViT/4	-	-	-	2B	555	2.65	278.1
SiD2 [26], UViT/4	-	-	-	N/A	137	2.19	-
SiD2 [26], UViT/2	-	-	-	N/A	653	1.48	-
PixNerd [70], XL/16	-	-	DINOv2	700 M	583	2.84	245.6
JiT-B/32	-	-	-	133M	26	4.02	271.0
JiT-L/32	-	-	-	462M	89	2.53	299.9
JiT-H/32	-	-	-	956M	183	1.94	309.1
JiT-G/32	-	-	-	2B	384	1.78	306.8

- High-resolution generation on pixels.

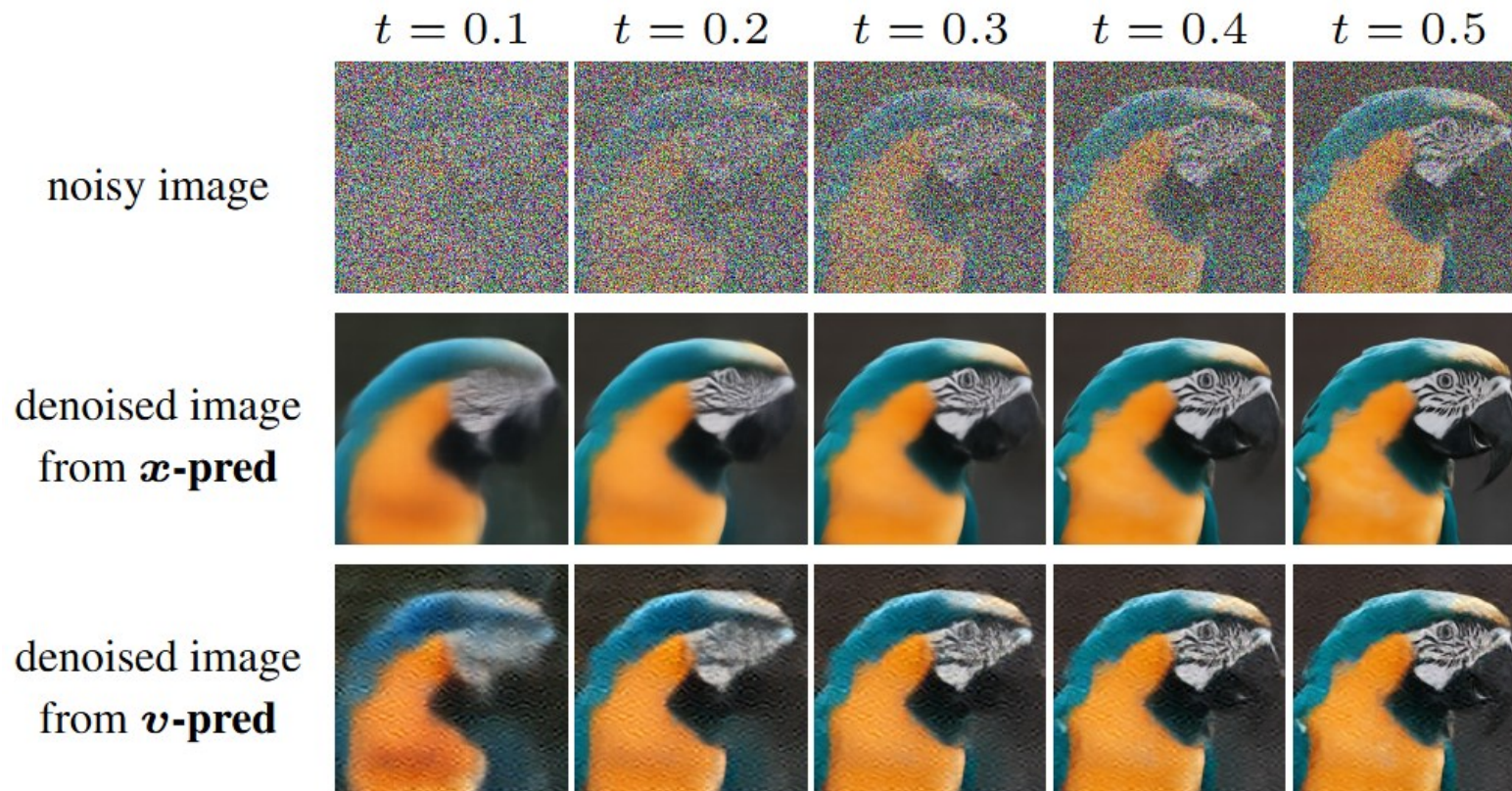


Figure 5. **Qualitative Results.** Selected examples on ImageNet 512×512 using JiT-H/32. More uncurated results are in appendix.

Noise is inherently different from natural data.

- 지난 수년간 diffusion model의 발전은 주로 probabilistic formulation에 초점을 맞추어 왔으며, 이에 비해 실제로 사용되는 신경망의 성능과 그 한계에 대해서는 상대적으로 적은 관심이 기울여졌다.
- 그러나 **신경망의 표현 능력은 무한하지 않으며**, 제한된 모델 용량을 노이즈를 모델링하는 데 사용하는 것보다 **실제 데이터를 모델링하는 데 사용하는 편이 더 효율적**일 수 있다.
 - 이러한 관점에서 보면, x -prediction의 효과는 돌이켜보면 자연스러운 결과라고 할 수 있다.
- 본 연구는 minimalist+self-contained 설계를 채택하여 domain-specific inductive bias를 줄임으로써, tokenizer를 얻기 어려운 다른 도메인에도 일반화될 수 있기를 기대한다.